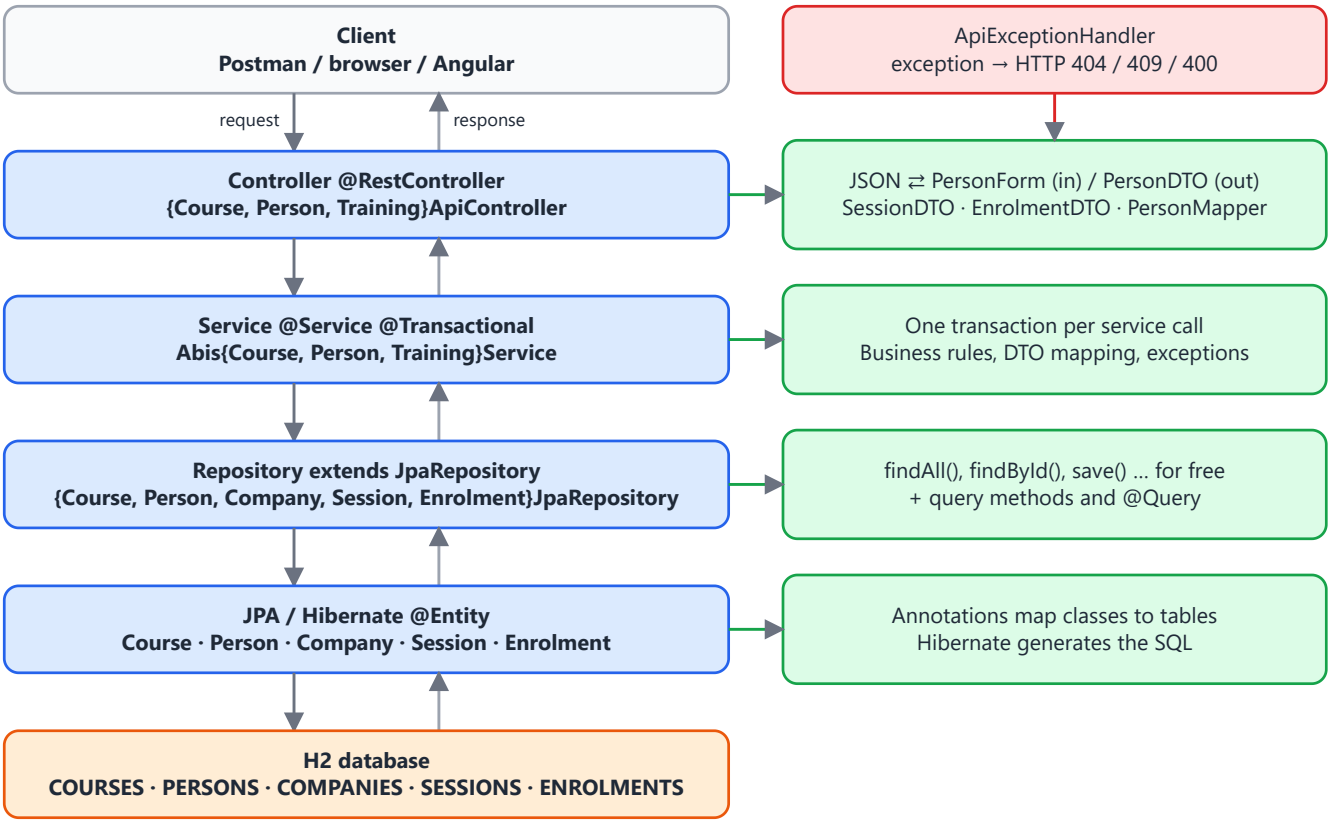
# Spring Data JPA in one picture

Visual companion to the JPA exercises (chapters 9–13) · solution project `JpaSolution`

## 1. The layers of the application

A request travels top to bottom; the answer travels back up. Each layer has exactly one job. The exercises replace the **bottom two layers** (a CSV file and an in-memory list) by a real database.

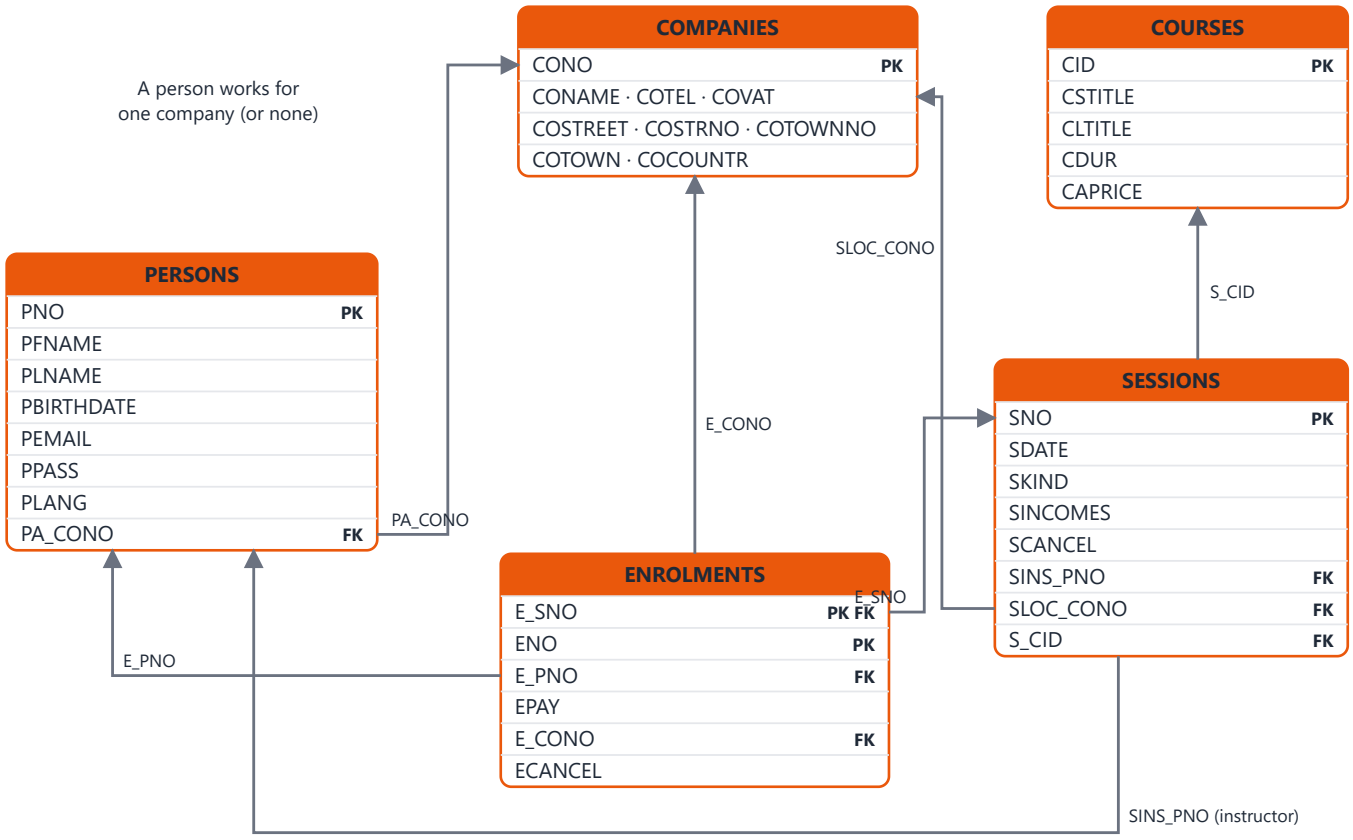



Java / Spring  Database  Error handling

> **Rule of thumb:** the controller never talks to a repository directly, and entities never leave the service layer as JSON (from chapter 12 on, DTOs and forms are used instead).

## 2. The database (what the entities must mimic)

Five tables. An arrow points from the table that holds the **foreign key** to the table it refers to (many → one).

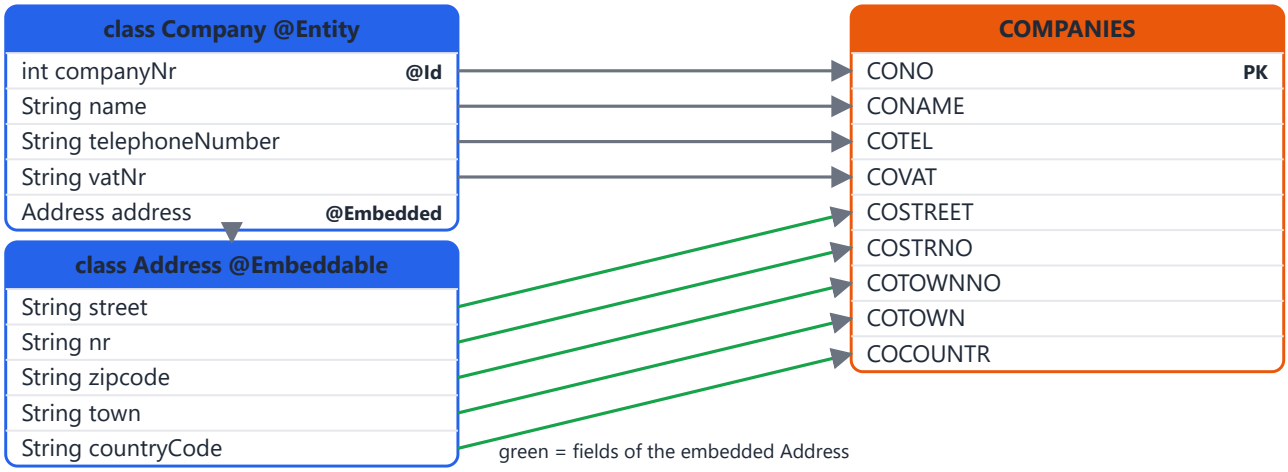



## 3. Annotation cheat sheet

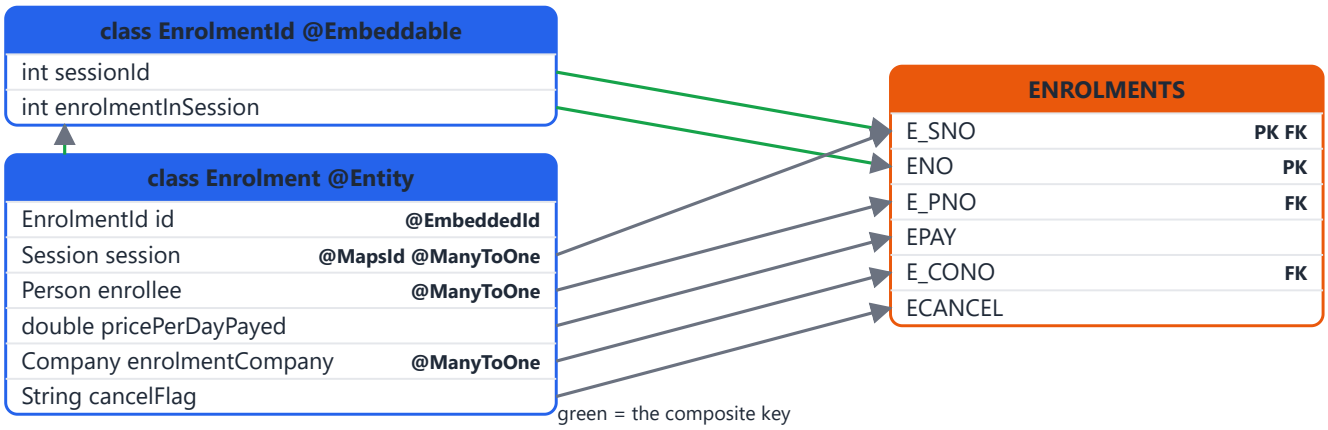
| Annotation | Meaning | Used in the solution |
|---|---|---|
| `@Entity` `@Table(name=..)` | This class is stored in that table | all model classes |
| `@Id` `@GeneratedValue(IDENTITY)` | Primary key, filled by the database (auto-increment) | `Course.courseId`, `Person.personId` |
| `@Column(name=..)` | Property ↔ column when the names differ | `shortTitle` ↔ `CSTITLE` |
| `@ManyToOne` `@JoinColumn` | Many rows point to one row via a foreign key column | `Person.company` → `PA_CONO` |
| `cascade = PERSIST` | Saving the person also inserts a new company | `Person.company` |
| `@Embeddable` / `@Embedded` | Value object stored in the *same* table | `Address` in `COMPANIES` |
| `@EmbeddedId` `@MapsId` | Composite primary key | `Enrolment (E_SNO + ENO)` |

## 4. From class to table — two mappings that need a closer look

**a) One table, two classes** — `Address` is not an entity: its fields become extra columns of `COMPANIES`.



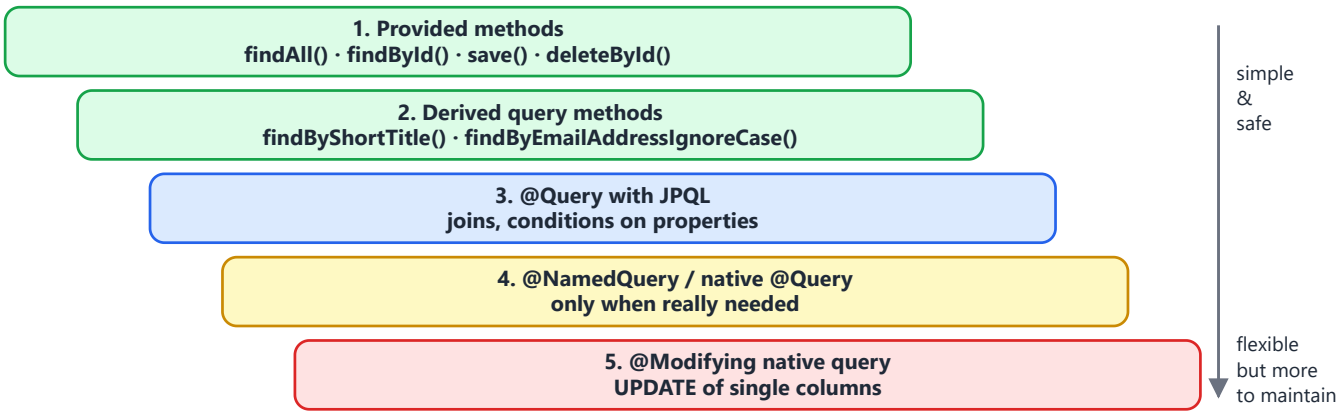

**b) Composite primary key** — an enrolment is identified by *session number + enrolment number in that session*. The key columns live in a small `@Embeddable` class, and `@MapsId` says that the `session` association supplies the `E_SNO` part.




> **Cancelled?** The database stores a letter ( `'C'` ) or `NULL` in `SCANCEL` / `ECANCEL` . The solution maps it as a `String cancelFlag` and derives `isCancelled()` from it, so queries can simply say `where scancel is null` .

## 5. Five ways to query

Start with the simplest one that works. Every repository extends `JpaRepository<Entity, IdType>`, which already gives `findAll()`, `findById()`, `save()`, `deleteById()` … without writing any code.

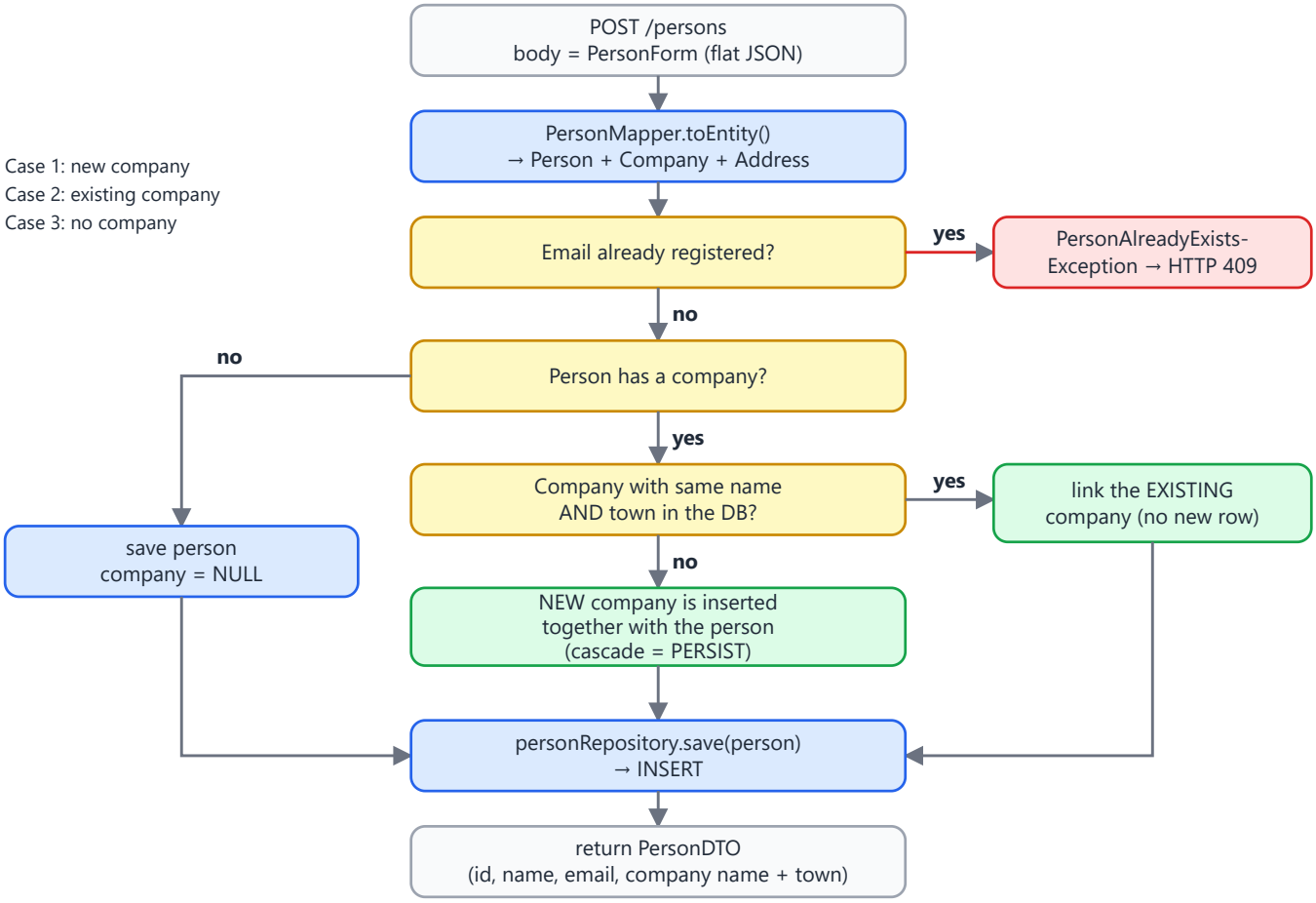



| Kind | Example from the solution | What it does |
|---|---|---|
| Derived query | `findByShortTitle(String t)`<br>`findByCompanyNameIgnoreCase(String n)` | Spring builds the SQL from the method name; `CompanyName` follows `person.company.name` |
| JPQL `@Query` | `select c from Company c where upper(c.name)=upper(:name) and upper(c.address.town)=upper(:town)` | Query on **classes and properties**, not on tables and columns |
| Native `@Query` | `select s.* from sessions s join courses c on s.s_cid=c.cid where ... and s.scancel is null` | Plain SQL, returns entities; needed for DB-specific SQL |
| `@NamedQuery` | `Enrolment.findEnrolmentsForPerson` on the entity, repository method `findEnrolmentsForPerson(personId)` | Query defined once on the entity; the repository method name must match |
| `@Modifying` native | `update sessions set scancel='C' where sno=:id`<br>`update courses set caprice=:price where cid=:id` | Changes only *one column*, returns the number of rows; **needs a transaction** |

> **JPQL vs SQL:** `from Company c where c.address.town = :t` (JPQL, property path) ↔ `from companies where cotown = ?` (SQL, column). Hibernate translates the first into the second.

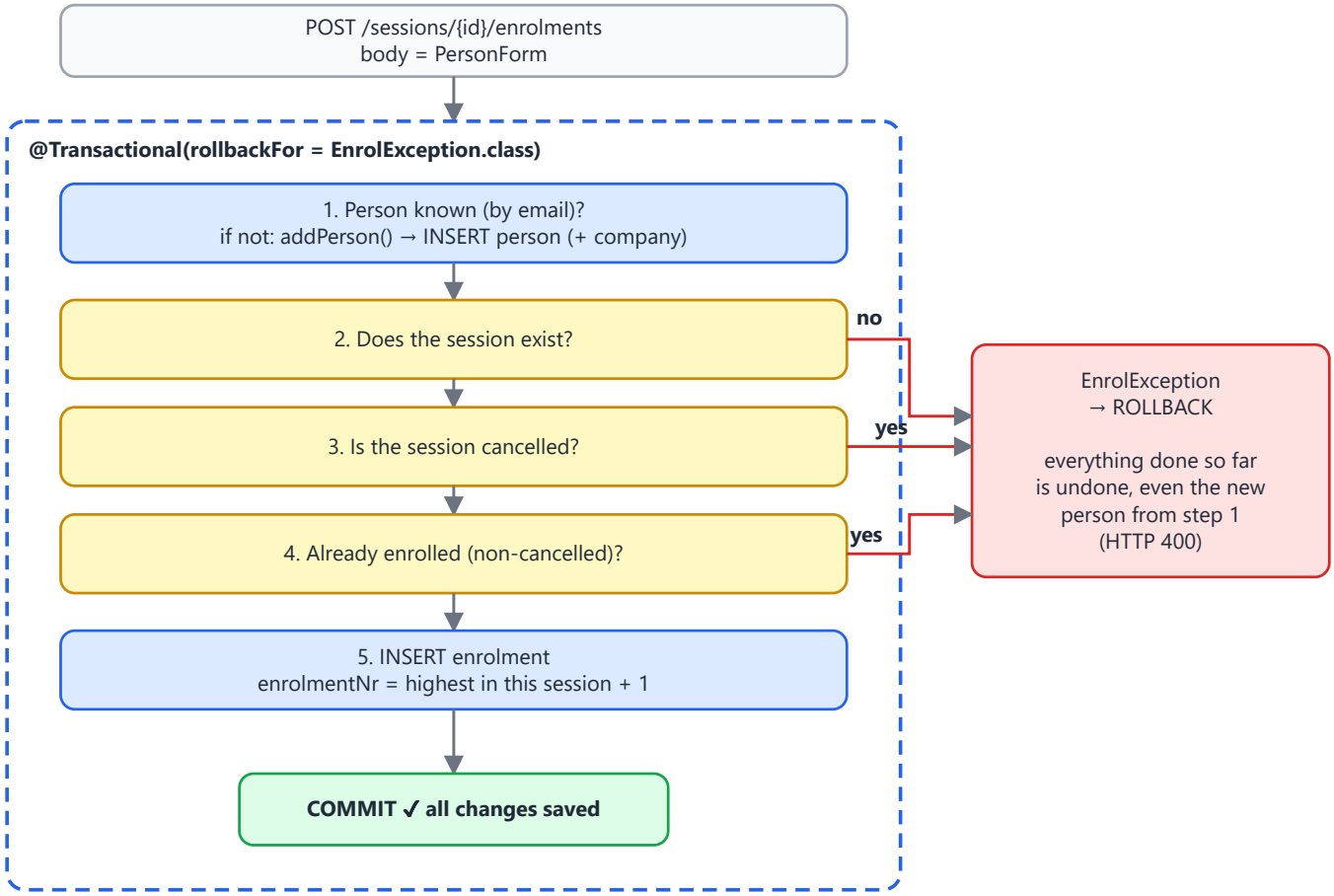
## 6. Adding a person (exercise 11.2)

The same service method handles all three cases from the exercise: new company, existing company, no company.



## 7. Enrolling for a session — transaction and rollback (exercise 13.5)

All steps run in **one transaction**. If any check fails, an `EnrolException` is thrown and the database is put back exactly as it was — including a person that was inserted in step 1.




**Testing tip.** A test method marked `@Transactional` is rolled back automatically after it finishes, so the test data never reaches the database and tests stay independent (no `@Order` needed). Tests that exercise the transaction of the service itself (cancel, enrol) must *not* be `@Transactional`, so they clean up after themselves.

## 8. Which exercise lives where?

| Exercise | Task | Classes in `JpaSolution` |
|---|---|---|
| 9.1–9.3 | Dependencies, H2 settings, test the connection, create the DB | `pom.xml`, `application.properties`, `createH2DB.sql`, `DataSourceTest` |
| 10.1–10.2 | Map `Course`, replace the course repository, DML tests | `Course`, `CourseJpaRepository`, `AbisCourseService`, `CourseServiceTest` |
| 11.2 | Map `Person`, add person (new / existing / no company) | `Person`, `PersonJpaRepository`, `AbisPersonService`, `PersonServiceTest` |
| 11.3, 12.3 | Sessions of a course (JPQL, then native without cancelled) | `Session`, `SessionJpaRepository`, `AbisTrainingService` |
| 11.1 | Company + Address, find by name / name and town | `Company`, `Address`, `CompanyJpaRepository`, `CompanyRepositoryTest` |
| 12.1 | PersonDTO, PersonForm, PersonMapper | `dto/PersonDTO`, `form/PersonForm`, `mapper/PersonMapper` |
| 12.2 | SessionDTO as a record | `dto/SessionDTO` |
| 12.4 | Enrolment: composite key, named query, EnrolmentDTO | `Enrolment`, `EnrolmentId`, `EnrolmentJpaRepository`, `dto/EnrolmentDTO` |
| 13.1 | PersonAlreadyExistsException works again | `AbisPersonService.addPerson`, `ApiExceptionHandler` |
| 13.2 | Delete a session unless it has enrolments | `AbisTrainingService.deleteSession`, `SessionCanNotBeDeletedException` |
| 13.3 | Change a course price with a modifying native query | `CourseJpaRepository.changePrice` |
| 13.4 | Cancel a session with a modifying native query | `SessionJpaRepository.cancelSession` |
| 13.5 | Transactional `enrolForSession` with rollback | `AbisTrainingService.enrolForSession`, `TrainingServiceTest` |

## 9. What changed compared to the starting project

| Before | After |
|---|---|
| `MemoryCourseRepository` (hard-coded list) | `CourseJpaRepository` — an interface, Spring writes the implementation |
| `FilePersonRepository` (CSV file, loops) | `PersonJpaRepository` — derived queries |
| Empty `addCourse` / `updateCourse` / `deleteCourse` | Real DML, annotated `@Transactional`; add and update return the saved object |
| Controllers returned the model classes | Controllers return DTOs and accept forms; a mapper converts |
| Training service methods returned `null` | Sessions, enrolments, cancel, delete and enrol are implemented |
| No database | H2 file database, `spring.jpa.show-sql=true` shows every generated statement in the console |

Run `mvnw test` to check the whole solution (45 tests). Start the app and call `http://localhost:8085/api/courses/7900` to see JSON coming out of the database.